Journal of Computational Intelligence and Decision Science

GNOSIS PRESS

ARTICLE

Enhanced LSTM-Based Recurrent Framework with Batch Normalization for Noise Reduction in Medical Imaging

Kiruthika R[1,*], Anitha G[2], Raju Rajkumar[3], and Rajkumar Palaniappan[4]

[1]Department of Computer Science, PSGR Krishnammal College for Women, Coimbatore, Tamilnadu, India.
[2]Assistant Professor, Department of Data Analytics (PG), PSGR Krishnammal College for Women, Coimbatore, Tamilnadu, India.
[3]Assistant Professor, Department of Computer Science, Pravabati College, Mayang Imphal, Manipur, India.
[4]Associate Professor, Department of Mechatronics Engineering, University of technology Bahrain, Bahrain.
[*]Corresponding Author: Kiruthika R. Email: kirthikamole@gmail.com

Received: 30 July 2025; Accepted: 25 October 2025; Published: 22 December 2025

ABSTRACT: The denoising of medical images corrupted by noise is a long-established problem in signal or image processing. This paper proposes an effective denoising scheme to remove white, salt, and pepper noises by combining Long Short-Term Memory (LSTM) based Batch Normalization and Recurrent Neural Network (RNN) techniques. In this study, the lung CT images are taken as input. Then, an effective batch size is computed by using Particle Swarm Optimization (PSO). Finally, the RNN algorithm is proposed for denoising the image. In RNNs, LSTM-based Batch Normalization was introduced to reduce internal covariate shift in neural networks. The performance is evaluated in terms of Peak Signal to Noise Ratio (PSNR) and Mean Squared Error (MSE). The experimental results proved that this algorithm is competitive with other denoising schemes.

KEYWORDS: Image denoising, Recurrent Neural Network, Long Short-Term Memory, Batch Normalization.

1 Introduction

With the development of imaging, computing, and communication technologies, there has been a rapid growth in image and video applications in recent years. At the same time, there is an increasing demand for high image quality. However, the signal captured by the camera is susceptible to noise during the acquisition and transmission process. Therefore, denoising remains an important problem in many image processing tasks and has attracted much research interest in the past decades [1].

A variety of image denoising methods have been developed in the past decades, including filtering-based methods, diffusion-based methods, total variation-based methods, wavelet/ curvelet-based methods, sparse representation-based methods, non-local self-similarity (NSS)-based methods, etc. [2]. Among these methods, the NSS models are popular in state-of-the-art methods. They perform good denoising, especially for images with regular and repetitive textures. This is because non-local means-based methods are usually better on images with regular and repetitive textures, whereas discriminative training-based methods generally produce better results on images with irregular textures or smooth areas. However, the NSS models inevitably suffer from two major drawbacks. First, they often need to specify a particular feature for denoising, which results in the algorithms maybe do not implementing good denoising jobs for all kinds

of images. Second, the models in general are nonconvex and involve several manually chosen parameters, providing some leeway to boost denoising performance [3].

To solve these problems, several discriminative learning methods have been recently developed to learn image prior models, such as the Trainable Non-Linear Reaction Diffusion (TNRD) model. The TNDR model is implemented by unfolding a fixed number of gradient descent inference steps. However, the TNDR model is inherently restricted to the specified forms of the priors. To be specific, the priors adopted in the TNRD are based on the analysis model, which is limited in capturing the full characteristics of image structures [4].

The TNRD model is a kind of deep neural network and can be expressed as a feed-forward deep network. There are some other deep neural networks for image denoising. Deep neural networks were used for image denoising. They claimed that deep neural networks have a similar or even better representation power than the Markov random field model on image denoising. Burger et al. proposed to use the multi-layer perceptron (MLP) for image denoising. Further, they combined sparse coding and deep neural networks pre-trained with a denoising auto-encoder. Although these deep neural networks achieve good performance on image denoising, these networks do not effectively explore the inherent characteristics of the image since they cascade the MLP-like networks [5].

Nowadays, Convolution Neural Network (CNN) has attracted more researchers because it has good abilities of self-learning through a large amount of data and does not need to strictly select the characteristics, only needs to guide learning to achieve the desired purpose. It is widely applied in the fields of image preprocessing, such as image super-resolution. Owing to the success in image super-resolution, some researchers have made attempts to apply the Deep CNN to image denoising. Lefkimmiatis proposed a novel deep network architecture for greyscale and colour image denoising that is based on a non-local image model. As the author claimed, the proposed method is indeed an NSS method, which utilizes the Deep CNN to learn the parameters of the NSS method. The Deep CNN was implemented for the denoising task, in which a contaminated image is input into the Deep CNN, and the corresponding latent clean image is output. Also, the stochastic gradient descent method is used as the training strategy, which will consume much time in training. But the important issue in convolutional architecture was proper depth computation, and the CNN models are only used to perform image denoising with additive white Gaussian noise. It does not consider real complex noises. So, the performance of this classifier needs to improve [6].

To solve this problem, LSTM-based Batch Normalization with Recurrent Neural Network (RNN) is proposed for denoising the images. First, the input image is processed. Then, we find that residual learning and batch normalization can greatly benefit the RNN learning as they can not only speed up the training but also boost the denoising performance. After that, the latent clean image can be achieved by separating the predicted noise image from the contaminated image. Further, we discuss the influence of the depth of our network on the denoising performance. Also, some network parameters are discussed. Finally, we conducted a series of comparison experiments to validate the proposed image denoising method.

This paper is organized as follows: In Section 2, the existing denoising schemes are discussed. Section 3 specifies the implementation of the RNN-based denoising in detail. In Section 4, the experimental results and discussions are presented. Conclusions are drawn in Section 4.

2 Related Works

In this section, image denoising based on existing schemes and their merits and demerits has been discussed. Zhang et al. [7] proposed a novel image denoising method based on a deep convolutional neural network (DCNN). Different from other learning-based methods, the authors design a DCNN to detect noise in images. Thus, the latent clear image can be achieved by separating the noise image from the contaminated image. At the training stage, the gradient clipping scheme is employed to prevent gradient explosions and enable the network to converge quickly. Experimental results demonstrate that the proposed denoising method can achieve a better performance compared with the state-of-the-art denoising methods. Also, the

results indicate that the denoising method could suppress different noises with different noise levels by means of a single denoising model.

Luo et al. [8] proposed an adaptive learning procedure to learn patch-based image priors for image denoising. The new algorithm, called the expectation-maximization (EM) adaptation, takes a generic prior learned from a generic external database and adapts it to the noisy image to generate a specific prior. Different from existing methods that combine internal and external statistics in ad hoc ways, the proposed algorithm is rigorously derived from a Bayesian hyper-prior perspective. There are two contributions of this paper. First, we provide a full derivation of the EM adaptation algorithm and demonstrate methods to improve the computational complexity. Second, in the absence of the latent clean image, we show how EM adaptation can be modified based on pre-filtering. The experimental results show that the proposed adaptation algorithm yields consistently better denoising results than the one without adaptation and is superior to several state-of-the-art algorithms.

Xiong et al. [9] proposed a new image denoising algorithm based on adaptive signal modeling and regularization. It improves the quality of images by regularizing each image patch using band-wise distribution modeling in the transform domain. Instead of using a global model for all the patches in an image, it employs content-dependent adaptive models to address the non-stationarity of image signals and the diversity among different transform bands. The distribution model is adaptively estimated for each patch individually. It varies from one patch location to another and varies for different bands. We consider the estimated distribution to have a non-zero expectation. Irrelevant patches are excluded so that such an adaptively learned model is more accurate than a global one. The image is ultimately restored via band wise adaptive soft-thresholding, based on a Laplacian approximation of the distribution of similar-patch group transform coefficients. Experimental results demonstrate that the proposed scheme outperforms several state-of-the-art denoising methods in both the objective and the perceptual qualities.

Liu et al. [10] proposed a Convolutional Neural Network (CNN) for image denoising, which achieves the state-of-the-art performance. Second, we propose a deep neural network solution that cascades two modules for image denoising and various high-level tasks, respectively, and use the joint loss for updating only the denoising network via back-propagation. We demonstrate that on the one hand, the proposed denoiser has the generality to overcome the performance degradation of different high-level vision tasks. On the other hand, with the guidance of high-level vision information, the denoising network can generate more visually appealing results. To the best of our knowledge, this is the first work investigating the benefit of exploiting image semantics simultaneously for image denoising and high-level vision tasks via deep learning.

Godard et al. [11] used the burst-capture strategy and implemented the intelligent integration via a recurrent fully convolutional deep neural net (CNN). We build our novel, multi-frame architecture to be a simple addition to any single-frame denoising model, and design it to handle an arbitrary number of noisy input frames. We show that it achieves state-of-the-art denoising results on our burst dataset, improving on the best published multi-frame techniques, such as VBM4D and FlexISP. Finally, we explore other applications of image enhancement by integrating content from multiple frames and demonstrate that our DNN architecture generalizes well to image super-resolution.

Remez et al. [12] demonstrated how the reconstruction quality improves when a denoiser is aware of the type of content in the image. To this end, we first propose a new fully convolutional deep neural network architecture, which is simple yet powerful as it achieves state-of-the-art performance even without being class-aware. We further show that a significant boost in performance of up to 0.4 dB PSNR can be achieved by making our network class-aware, namely, by fine-tuning it for images belonging to a specific semantic class. Relying on the hugely successful existing image classifiers, this research advocates for using a class-aware approach in all image enhancement tasks.

Zhang et al. [13] took one step forward by investigating the construction of feed-forward denoising convolutional neural networks (DnCNNs) to embrace the progress in very deep architecture, learning algorithm, and regularization method into image denoising. Specifically, residual learning and batch

normalization are utilized to speed up the training process as well as boost the denoising performance. Different from the existing discriminative denoising models, which usually train a specific model for additive white Gaussian noise (AWGN) at a certain noise level, our DnCNN model is able to handle Gaussian denoising with unknown noise level (i.e., blind Gaussian denoising). With the residual learning strategy, DnCNN implicitly removes the latent clean image in the hidden layers. This property motivates us to train a single DnCNN model to tackle several general image denoising tasks, such as Gaussian denoising, single-image super-resolution, and JPEG image deblocking. Our extensive experiments demonstrate that our DnCNN model can not only exhibit high effectiveness in several general image denoising tasks, but also be efficiently implemented by benefiting from GPU computing.

Sato et al. [14] proposed a new image denoising method using multiple-minimum cuts based on the maximum-flow neural network (MF-NN), which is our proposed minimum cut algorithm based on the nonlinear resistive circuit analysis. The MF-NN has two unique features not shared by the conventional minimum cut algorithm. One is that multiple minimum cuts can be obtained simultaneously, and the other is that it is suitable for hardware implementation. By using the MF-NN's features, we find novel solutions for two issues of the conventional graph cuts.

Al-Sbou [15] presented a detailed performance evaluation of using neural networks as a noise reduction tool. The proposed approach includes using both mean and median statistical functions for calculating the output pixels of the training pattern of the neural network. This uses part of the degraded image pixel to generate the system training pattern. Different test images, noise levels, and neighborhood sizes are used. Based on using samples of degraded pixel neighborhoods as inputs, the output of the proposed approach provided a good image denoising performance, which exhibited promising qualitative and quantitative results of the degraded noisy images in terms of PSNR, MSE, and visual tests.

3 Proposed Methodology

In this section, the proposed LSTM-based batch normalization with an RNN-based image denoising process has been discussed.

3.1 System Overview

Initially, the lung CT images are taken from the SIMBA database and processed, and then the RNN is proposed to denoise the image. In this RNN process, the LSTM with batch normalization is presented to improve the PSNR of the proposed scheme. In batch normalization, the batch size is optimally selected by using PSO. Experimental results show that the proposed RNN attained better performance compared to other denoising schemes.

$$y(i,j) = x(i,j) + n(i,j) \quad (1)$$

Where y(i,j) is the observed value, x(i,j) is the true (original) value, and n(i,j) is the noise perturbation at pixel (i,j). The learning phase of the MLP attempts to make it capture inherent space relations of degraded pixels and correspond them to the non-degraded pixels. The degraded image data is provided as input to the MLP, and the non-degraded image as the corresponding output in the supervised learning process.

3.2 Batch Normalization

In optimization, whitening is a common procedure that has been shown to reduce convergence rates. Extending the idea to deep neural networks, one can think of an arbitrary layer as receiving samples from a distribution that is shaped by the layer below. This distribution changes during the course of training, making any layer but the first responsible not only for learning a good representation but also for adapting to a changing input distribution. This distribution variation is termed Internal Covariate Shift, and reducing it is hypothesized to help the training procedure.

To reduce this internal covariate shift, we could whiten each layer of the network. However, this often turns out to be too computationally demanding. Batch normalization approximates the whitening by standardizing the intermediate representations using the statistics of the current minibatch. Given a mini-batch x, we can calculate the sample mean and sample variance of each feature k along the mini-batch axis.

$$\bar{X}_k = \frac{1}{m}\sum_{i=1}^{m} x_{i,k} \tag{2}$$

$$\sigma_k^2 = \frac{1}{m}\left(x_{i,k} - \bar{x}_k\right)^2 \tag{3}$$

Where m is the size of the mini-batch. The mini-batch is optimally selected by using PSO to improve the PSNR performance in RNN.

3.3 Particle Swarm Optimization

Total PSO proposed by Kennedy and Eberhart. The PSO algorithm is annoyed by the social behavior of a group of migrating birds trying to reach an indefinite destination. Each solution is termed as a 'bird' in the flock and is known as a 'particle'. A particle corresponds to a chromosome in Genetic Algorithms (GAs). Not like GAs, the evolutionary procedure in the PSO does not create new birds from parent ones. Instead, the birds in the population only build up their social behavior and resulting in their movement towards a destination.

In this study, the objective function of this PSO is to optimally select the batch size in the RNN for training. A group of birds communicates when they fly. Each bird appears in a particular direction, and they communicate collectively and recognize the bird that is in the best location. Consequently, each bird speeds in the direction of the best bird through a velocity that is based on its current position. All birds inspect the search space from their new local location, and the process is repeated until the flock arrives at a favoured destination. It is to be observed that the procedure comprises both social interaction and intelligence, so that the birds discover from their own experience, called local search, and also from the experience of others around them, called global search.

The process is initiated with a collection of random particles, N. The i^{th} particle is denoted by its position as a point in S-dimensional space, where S denotes the number of variables. During the process, each particle i observes three values, namely its current position $(X_{i)}$, the best position it arrived in previous cycles $(P_{i)}$, its flying velocity (V_i). These three values are denoted as follows:

Current position $X_i = (x_{i1}, x_{i2}, \dots, x_{iS})$

Best previous position $P_i = (p_{i1}, p_{i2}, \dots, p_{iS})$ (4)

Flying velocity $V_i = (v_{i1}, v_{i2}, \dots, v_{iS})$

In each time interval (cycle), the position $(P_g)$of the best particle (g) is computed as the best fitness of all particles.

Thus, each particle updates its velocity V_i to get closer to the best particle g, as follows.

$$New \quad V_i = \omega \times current \;\; V_i + c_1 \times rand() \times (P_i - X_i) + c_2 \times Rand() \;\; \times (P_i - X_i) \tag{5}$$

As such, using the new velocity V_i, the particle's updated position becomes:

$$New \;\; position \;\; X_i = current \;\; position \;\; X_i + \;\; New \;\; V_i \tag{6}$$

$$V_{max} \geq V_i \geq -V_{max} \tag{7}$$

where c_1 and c_2 represent two positive constants named learning factors (usually $c_1 = c_2 = 2$); rand ( ) and Rand ( ) denote two random functions in the range [0, 1], V_{max} is an upper limit on the maximum change of particle velocity, ω denotes an inertia weight employed as an enhancement to manage the influence of the previous history of velocities on the current velocity. The ω balances the global search and the local search, and it is introduced to minimize linearly with time from a value of 1.4–0.5. For this, itself global search itself initiates with a large weight and then decreases with time to favor local search over global search.

It is observed that the second term in equation (6) indicates cognition or the private judgment of the particle when compared with its current position to its own best position. The third term in equation (6) denotes the social collaboration between the particles and compares a particle's current position to that of the best particle. Furthermore, to control the change that occurs in the particle velocities, upper and lower bounds for velocity change are limited to a user-specified value of V_{max}. Once the new position of a particle is computed using equation (5), the particle then flies towards it. Therefore, the main parameters used in the PSO are the population size (number of birds), the number of generation cycles, and the maximum change of a particle's velocity V_{max} and ω.

The random numbers of nodes are initialized, and they are denoted by N or k. Compute the relative weight of edges G by initializing the value of weight factor ω, then calculate the fitness function for all the nodes. For each distance of the sensor node, a best position is determined as a $pBest$. If fitness (i) is better than $pBest$, $pBest(i) = fitness\ (i)$ function is satisfied, and the execution ends, the data is stored in the nodes. If the condition is not satisfied, assign an energy cost to the fitness value, then calculate the temporary energy cost for each node. When the calculated temporary energy cost and the energy cost are equal to each other, the algorithm ends, and the data is stored successfully in the nodes for future retrieval process.

Detailed pseudo-code of the PSO algorithm:

1. A population of agents is created randomly.

$$X_i = (P_1, P_2, P_3, \dots\dots\dots, P_N) \tag{8}$$

2. Each particle's position according to the objective function is evaluated. In this case it is the total operational cost given by C for each particle and evaluate their fitness (i.e minimization of the objective function)
3. Cycle =1
4. Repeat
5. Update the velocity of the particles according to the formula

$$V_i(t) = V_i(t-1) + C_i r_i\big(pbest(t) - x_i\ (t-1)\big) + C_2 r_2\big(gbest(t) - x_i\ (t-1)\big) \tag{9}$$

c = acceleration factor. r = random values between 1 and 0

6. Evaluate the velocity to ascertain if it is the range of

$$V_{max} \le V_i \le V_{min} \tag{10}$$

7. Move particles to their new position

$$X_i(t) = X_i(t-1) + V_i(t) \tag{11}$$

8. Evaluate to ensure that limits have not been exceeded.
9. Compare the particle's fitness evaluation with its previous pbest. If the current value is better than the previous pbest, then set the pbest value equal to the current value and the pbest location equal to the current location in the N dimensional search space.

10. Compare the best current fitness evaluation with the population gbest. If the current value is better than the population gbest, then reset the gbest to the current best position and the fitness value to current fitness value.

11. Check if stopping criterion had been met. If not update the cycle and go back to step (5).

12. End when the stopping criterion, which here is the number of iterations, has been met.

Using these statistics, we can standardize each feature as follows

$$\hat{x}_k = \frac{x_k - \bar{x}_k}{\sqrt{\sigma_k^2 + \epsilon}} \tag{12}$$

Where ϵ is a small positive constant to improve numerical stability. However, standardizing the intermediate activations reduces the representational power of the layer. To account for this, batch normalization introduces additional learnable parameters γ and β, which respectively scale and shift the data, leading to a layer of the form

$$BN(X_k) = \gamma_k \hat{X}_k + \beta_k \tag{13}$$

By setting γ_k to σ_k and $\beta_k$to $\bar{x}_k$, the network can recover the original layer representation. So, for a standard feedforward layer in a neural network

$$y = \phi(Wx + b) \tag{14}$$

Where W is the weights matrix, b is the bias vector, xis the input of the layer and φ is an arbitrary activation function, batch normalization is applied as follows

$$y = \phi\big(BN(Wx)\big) \tag{15}$$

Note that the bias vector has been removed, since its effect is cancelled by the standardization. Since the normalization is now part of the network, the back propagation procedure needs to be adapted to propagate gradients through the mean and variance computations as well. At test time, we can't use the statistics of the mini-batch. Instead, we can estimate them by either forwarding several training mini-batches through the network or averaging their statistics, or by maintaining a running average calculated over each mini-batch seen during training.

3.4 Recurrent Neural Networks (RNNs)

RNN extends Neural Networks to sequential data. Given an input sequence of vectors $(x_1, \ldots, x_T)$ they produce a sequence of hidden states $(h_1, \ldots, h_{T)}$ which are computed at time step t as follows

$$h_t = \phi(W_h h_{t-1} + W_x x_t) \tag{16}$$

Where W_h is the recurrent weight matrix, W_x is the input-to-hidden weight matrix, and φ is an arbitrary activation function. If we have access to the whole input sequence, we can use information not only from the past time steps, but also from the future ones, allowing for bidirectional RNNs.

$$\vec{h}_t = \phi\big(\vec{W}_h \vec{h}_{t-1} + \vec{W}_x x_t\big) \tag{17}$$

$$\overleftarrow{h_t} = \phi(\overleftarrow{W}_h \overleftarrow{h}_{t-1} + \overleftarrow{W}_x x_t) \tag{18}$$

$$h_t = [\vec{h}_t : \overleftarrow{h_t}] \tag{19}$$

Where [x: y] denotes the concatenation of x and y. Finally, we can stack RNNs by using h as the input to another RNN, creating deeper architectures.

$$h_t^l = \phi\left(W_h h_{t-1}^l + W_x h_t^{l-1}\right) \tag{20}$$

In vanilla RNNs, the activation function φ is usually a sigmoid function, such as the hyperbolic tangent. Training such networks is known to be particularly difficult because of vanishing and exploding gradients.

3.5 Long Short-Term Memory

A commonly used recurrent structure is the Long Short-Term Memory (LSTM). It addresses the vanishing gradient problem commonly found in vanilla RNNs by incorporating gating functions into its state dynamics. At each time step, an LSTM maintains a hidden vector h and a cell vector c responsible for controlling state updates and outputs. More concretely, we define the computation at time step t as follows:

$$i_t = sigmoid(W_{hi} h_{t-1} + W_x x_t) \tag{21}$$

$$f_t = sigmoide\left(W_{hf} h_{t-1} + W_{hf} x_t\right) \tag{22}$$

$$c_t = f_t \odot c_{t-1} + i_t \tanh \odot (W_{hc} h_{t-1} + W_{xc} x_t) \tag{23}$$

$$o_t = sigmoide(W_{ho} h_{t-1} + W_{hx} x_t + W_{co} c_t) \tag{24}$$

$$h_t = o_t \odot \tanh(c_t) \tag{25}$$

Where sigmoid (•) is the logistic sigmoid function, tanh is the hyperbolic tangent function, W_h are the recurrent weight matrices and W_x are the input-to-hidden t weight matrices. i_t, f_t and o_t are respectively the input, forget, and output gates, and c_t is the cell.

3.6 Batch Normalization for RNNs

From equation 16, an analogous way to apply batch normalization to an RNN would be as follows:

$$h_t = \phi\left(BN(W_h h_{t-1} + W_x x_t)\right) \tag{26}$$

However, in our experiments, when batch normalization was applied in this fashion, it did not help the training procedure. Instead, we propose to apply batch normalization only to the input-to-hidden transition *($W_x x_t$)*, i.e., as follows:

$$h_t = \phi\left(BN(W_h h_{t-1} + W_x x_t)\right) \tag{27}$$

This idea is similar to the way dropout can be applied to RNNs: batch normalization is applied only on the vertical connections (i.e., from one layer to another) and not on the horizontal connections (i.e., within the recurrent layer). We use the same principle for LSTMs: batch normalization is only applied after multiplication with the input-to-hidden weight matrices W_x. Based on the above procedures, the images are normalized.

4 Results and Discussion

The proposed procedure requires no prior knowledge of the noise; this represents a good advantage of the proposed approach compared to many of the current techniques that assume the noise model to be known, like the Gaussian noise. That is because, in reality, this assumption may not always hold due to the varied nature and sources of noise. The experimental results presented here are based on adding white noise, salt and pepper noise with zero mean and different variance values to demonstrate the performance of the proposed algorithm. The proposed technique was implemented using MATLAB. The experimental work is performed on the SIMBA dataset. The database currently consists of an image set of 50 low-dose documented whole-lung CT scans for detection. The CT scans were obtained in a single breath hold with a 1.25 mm slice thickness. The locations of nodules detected by the radiologist are also provided. Figures 1, 2, and 3 images are taken from the SIMBA dataset. Peak to Signal Noise Ratio (PSNR) and MSE are standard criteria reported in the literature for quantitative evaluation of the effectiveness of proposed image denoising methods. PSNR and MSE are metrics by which they measure the absolute difference between two completely quantifiable signals. PSNR is the ratio between the reference signal and the distorted signal in an image, given in decibels. The higher the PSNR, the closer the distorted image is to the original. In general, a higher PSNR value should correlate to a higher quality image, but this is not always the case.

Figure 1: Input image 1

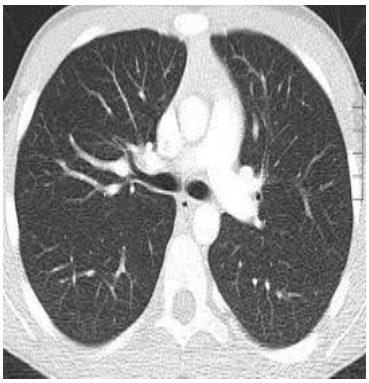

Figure 2: Input image 2

Figure 3: Input image 3

MSE is the average squared difference between a reference image and a distorted image. It is computed pixel-by-pixel by adding up the squared differences of all the pixels and dividing by the total pixel count. Both MSE and PSNR are very important in image and video quality monitoring and are computed as below,

$$MSE = \frac{1}{mn}\sum_{i=0}^{m-1}\sum_{j=0}^{n-1}[I(i,j) - K(i,j)]^2 \tag{28}$$

$$PSNR = 10.\log_{10}\left(\frac{MAX^2}{MSE}\right) \tag{29}$$

Where I and K are the original and the distorted images, respectively. m and n are the number of pixels in both images (dimensions of the images), and MAX is equal to the maximum possible pixel value.

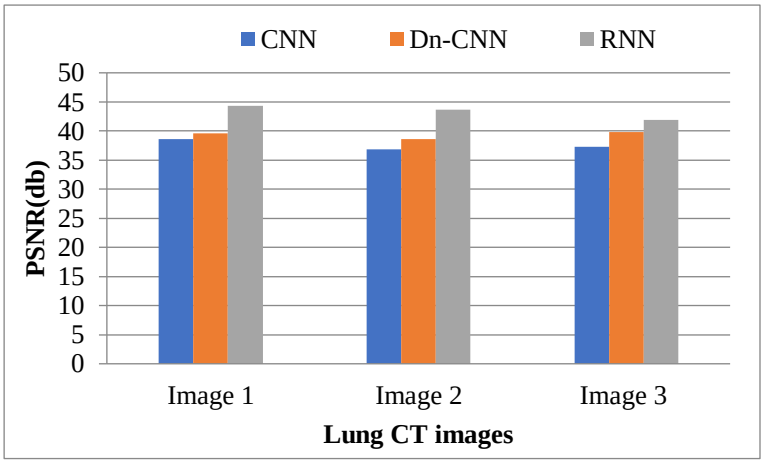

Figure 4: Experimental results of PSNR comparison for lung CT images

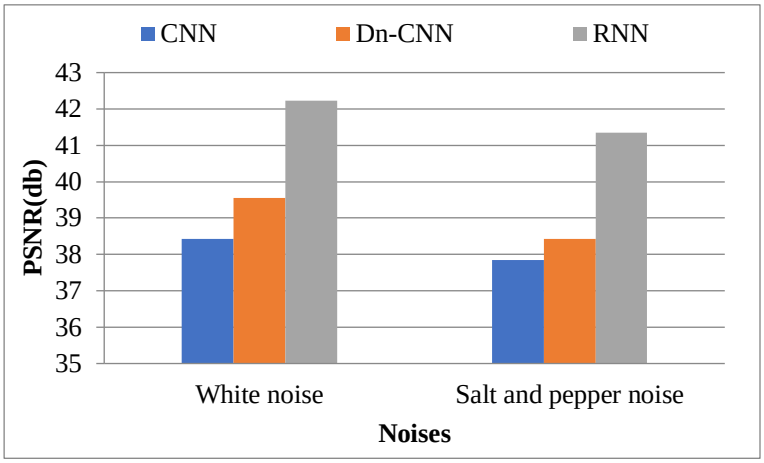

Figure 5: Experimental results of PSNR comparison against noise

Figure 4 illustrates the PSNR of the input images based on the mean function. From this figure, it can be noticed that as the input image increases, the PSNR increases. The performance of the proposed RNN scheme is compared with the existing image denoising schemes like DnCNN, CNN. The PSNR results of the proposed RNN schema are higher after the noise application when compared to existing denoising methods, as shown in Fig.4.

Fig. 5 illustrates the PSNR of the lung CT images based on the white, salt, and pepper noises. From this figure, it can be noticed that the proposed RNN attained high PSNR performance compared to existing image denoising schemes like DnCNN, CNN. Due to the optimal batch selection and batch normalization, the proposed RNN scheme attained better results.

Figure 6 illustrates the MSE of the lung CT images based on the mean function. From this figure, it can be noticed that as the input image increases, the MSE increases. The performance of the proposed RNN scheme is compared with the existing image denoising schemes like DnCNN, CNN. The MSE results of the proposed RNN schema are lower after the noise application when compared to existing denoising methods, as shown in Fig.6.

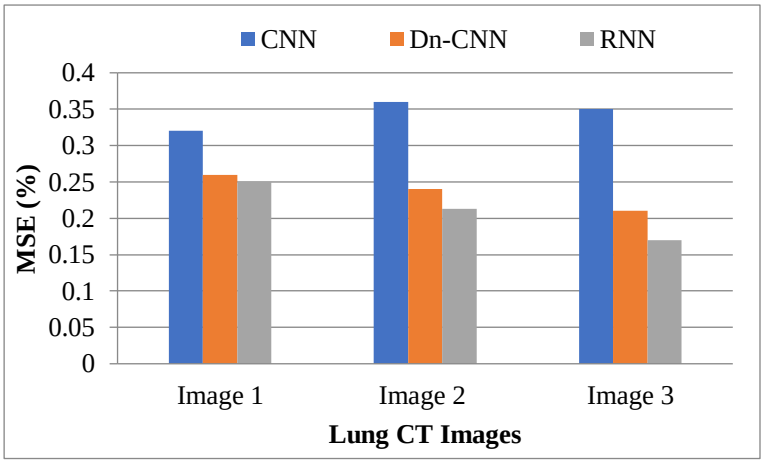

Figure 6: Experimental results of MSE comparison for lung CT images

Fig. 7 illustrates the MSE of the lung CT images based on the white, salt, and pepper noises. From this figure, it can be noticed that the proposed RNN attained a lower MSE error rate compared to existing

image denoising schemes like DnCNN, CNN. Due to the LSTM-based batch normalization, the proposed RNN scheme attained better results.

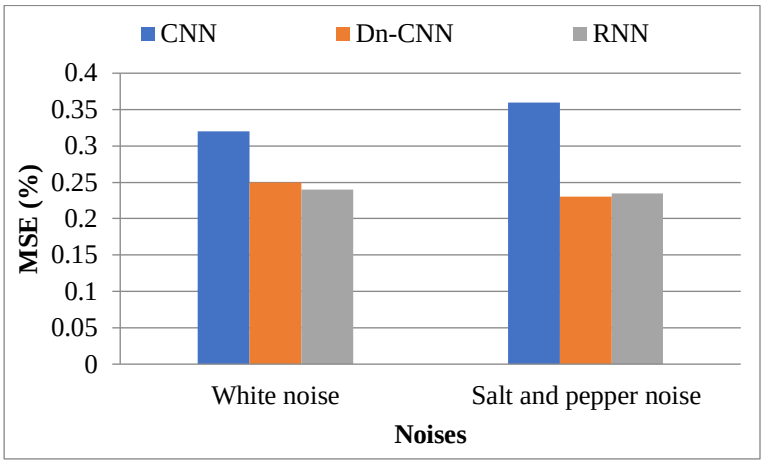

Figure 7: Experimental results of MSE comparison against noise

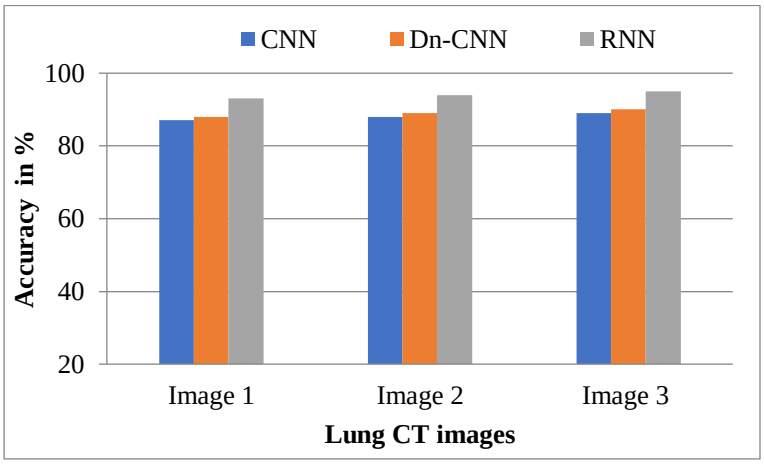

Figure 8: Experimental results of accuracy comparison for lung CT images

From this figure, it can be noticed that the proposed RNN attained high accuracy compared to existing image denoising schemes like DnCNN, CNN.

5 Conclusion

In this study, an effective denoising scheme is proposed to remove white, salt, and pepper noises by combining LSTM-based Batch Normalization and RNN techniques. In this study, first, the input images are processed. Then, an effective batch size is computed by using PSO. Finally, the RNN algorithm is proposed for denoising the image. In RNN, LSTM-based Batch Normalization was introduced to reduce internal covariate shift in neural networks. The PSNR and MSE results show that the proposed RNN attained better results compared to other denoising schemes like DnCNN and CNN. Comparison of experimental results verifies the good denoising ability of the proposed method and indicates that it provides an effective solution to image denoising. In the future, some other denoising algorithms will focus on improving the PSNR performance result.

Acknowledgement: Not applicable.

Funding Statement: Not applicable.

Author Contributions: The authors confirm contribution to the paper as follows: Conceptualization, methodology and writing—original draft preparation: Niranchana Radhakrishnan; supervision and writing: C. Viji; review and editing: Balusamy Nachiappan. All authors reviewed the results and approved the final version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest to report regarding the present study.

References

1. Chatterjee P, Milanfar P. Patch-based near-optimal image denoising. IEEE Trans Image Process. 2011;21(4):1635-49.
2. Zhang J, Xiong R, Zhao C, Ma S, Zhao D. Exploiting image local and nonlocal consistency for mixed Gaussian-impulse noise removal. In 2012 IEEE International Conference on Multimedia and Expo; 2012 Jul 9, Melbourne, VIC, Australia; 2012. p. 592-597.
3. Chen Y, Liu KR. Image denoising games. IEEE Trans Circuits Syst Video Technol. 2013;23(10):1704-16.
4. Rajwade A, Rangarajan A, Banerjee A. Image denoising using the higher order singular value decomposition. IEEE Trans Pattern Anal Mach Intell. 2012;35(4):849-62.
5. Mäkitalo M, Foi A. Noise parameter mismatch in variance stabilization, with an application to Poisson–Gaussian noise estimation. IEEE Trans Image Process. 2014;23(12):5348-59.
6. Zuo W, Zhang L, Song C, Zhang D, Gao H. Gradient histogram estimation and preservation for texture enhanced image denoising. IEEE Trans Image Process. 2014;23(6):2459-72.
7. Zhang F, Cai N, Wu J, Cen G, Wang H, Chen X. Image denoising method based on a deep convolution neural network. IET Image Process. 2018;12(4):485-93.
8. Luo E, Chan SH, Nguyen TQ. Adaptive image denoising by mixture adaptation. IEEE Trans Image Process. 2016;25(10):4489-503.
9. Xiong R, Liu H, Zhang X, Zhang J, Ma S, Wu F, Gao W. Image denoising via bandwise adaptive modeling and regularization exploiting nonlocal similarity. IEEE Trans Image Process. 2016;25(12):5793-805.
10. Liu D, Wen B, Liu X, Wang Z, Huang TS. When image denoising meets high-level vision tasks: A deep learning approach. arXiv preprint arXiv:1706.04284. 2017 Jun 14.
11. Godard C, Matzen K, Uyttendaele M. Deep burst denoising. In Proceedings of the European conference on computer vision (ECCV); 2018. p. 538-554.
12. Remez T, Litany O, Giryes R, Bronstein AM. Deep class-aware image denoising. In 2017 international conference on sampling theory and applications (SampTA); 2017 Jul 3, Tallinn, Estonia; 2017. p. 138-142.
13. Zhang K, Zuo W, Chen Y, Meng D, Zhang L. Beyond a gaussian denoiser: Residual learning of deep cnn for image denoising. IEEE Trans Image Process. 2017;26(7):3142-55.
14. Sato M, Otake T, Aomori H, Tanaka M. New image denoising method using multiple-minimum cuts based on maximum-flow neural network. In 2015 European Conference on Circuit Theory and Design (ECCTD); 2015 Aug 24, Trondheim, Norway; 2015. p. 1-4.
15. Al-Sbou YA. Artificial neural networks evaluation as an image denoising tool. World Appl Sci J. 2012;17(2):218-27.